

Python Trading Platform Interview Cookbook

Author: Amol Charpe

Project context: Alsamosa / IONE ALGO trading dashboard

Goal: Prepare for Python senior developer / team lead technical interviews.

How To Use This Book

Read this as a practical interview manual. You do not need to memorize every line, but you should be able to explain:

- * What the system does.
- * Why the architecture was chosen.
- * How live market data flows through the backend.
- * How replay is isolated from live trading.
- * How orders are placed safely.
- * How latency is reduced.
- * How to write core code if asked in interview.

Your one-minute project pitch:

Quote: I built an algo-trading platform using Python and FastAPI. It supports Dhan and Zerodha brokers, live websocket market data, stock and NIFTY modes, replay and bulk historical simulation, paper trading, order reconciliation, pyramiding, stop-loss management, and a hybrid mode where NIFTY acts as the driver and stocks act as the traded instruments. The architecture separates market data, candle building, strategy, order execution, replay, and dashboard state so that live ticks and user requests do not block each other.

Table Of Contents

1. Python Foundations
2. Object Oriented Programming
3. Data Structures And Algorithms For Trading
4. FastAPI Basics To Advanced
5. Databases And Persistence
6. Authentication And JWT
7. WebSockets And Market Data
8. Algo Trading Latency
9. Trading System Architecture
10. Order Manager And Reconciliation

11. Replay And Backtesting
12. Hybrid Mode Design
13. Multi-Client SaaS Design
14. Google Cloud Deployment
15. Code You Must Be Able To Write
16. Interview Questions And Answers

1. Python Foundations

Core Python Concepts

Python is dynamically typed, interpreted, object-oriented, and high-level. In backend trading applications we use it for:

- * API servers.
- * Broker SDK integration.
- * Strategy calculation.
- * Candle generation.
- * Replay simulation.
- * Background workers.

Important concepts:

- * Variables are references.
- * Lists and dictionaries are mutable.
- * Tuples, strings, integers, and floats are immutable.
- * Functions are first-class objects.
- * Exceptions should be used for abnormal flow, not ordinary business logic.
- * Type hints improve maintainability but are not enforced at runtime unless validated by libraries like Pydantic.

Example:

```
from dataclasses import dataclass
```

```
@dataclass
class Tick:
    symbol: str
    ltp: float
    volume: int
    timestamp: str
```

```
def normalize_symbol(symbol: str) -> str:
    return symbol.strip().upper()
```

```
tick = Tick(symbol=normalize_symbol(" tcs "), ltp=2150.5, volume=1000,
timestamp="2026-07-01 09:15:01")
print(tick.symbol)
```

Common Python Interview Questions

Question: What is the difference between list and tuple?

Answer: A list is mutable and used when data changes. A tuple is immutable and useful for fixed records or dictionary keys.

Question: What is a generator?

Answer: A generator yields values lazily and does not load all data into memory.

```
def candle_batches(candles, batch_size):
    for i in range(0, len(candles), batch_size):
        yield candles[i:i + batch_size]
```

Question: Why use type hints?

Answer: Type hints make large codebases safer. IDEs and static checkers can detect wrong calls before runtime.

2. Object Oriented Programming

OOP helps model real trading concepts:

- * BrokerClient
- * MarketDataFeed
- * CandleBuilder
- * StrategyEngine
- * OrderManager
- * Position
- * ReplayEngine

Encapsulation

Keep internal state private and expose methods.

```
class Position:
    def __init__(self, symbol: str, quantity: int, entry_price: float):
        self.symbol = symbol
```

```
self.quantity = quantity
self.entry_price = entry_price
self.open_quantity = quantity
```

```
def pnl(self, current_price: float) -> float:
    return round((current_price - self.entry_price) * self.open_quantity, 2)
```

Inheritance

Useful for broker adapters, but avoid deep inheritance.

```
from abc import ABC, abstractmethod

class BrokerAdapter(ABC):
    @abstractmethod
    def place_order(self, symbol: str, side: str, quantity: int) -> str:
        raise NotImplementedError

class DhanBroker(BrokerAdapter):
    def place_order(self, symbol: str, side: str, quantity: int) -> str:
        return "dhan-order-id"

class ZerodhaBroker(BrokerAdapter):
    def place_order(self, symbol: str, side: str, quantity: int) -> str:
        return "kite-order-id"
```

Composition

Prefer composition for production systems.

```
class TradingEngine:
    def __init__(self, feed, strategy, order_manager):
        self.feed = feed
        self.strategy = strategy
        self.order_manager = order_manager
```

Interview answer:

Quote: I prefer composition over inheritance in this project because broker feeds, strategy engines, and order managers are independently replaceable components.

3. Data Structures And Algorithms For Trading

Dictionaries For Symbol Lookup

```
sessions: dict[str, StockSession] = {}  
security_to_symbol: dict[str, str] = {}
```

Use dictionaries because symbol lookup must be $O(1)$.

Deque For Recent Candles

```
from collections import deque  
  
recent_ticks = deque(maxlen=1000)  
recent_candles = deque(maxlen=200)
```

Priority Selection: Top Gainer / Loser

```
def select_top_gainer(changes: dict[str, float]) -> str | None:  
    if not changes:  
        return None  
    return max(changes, key=changes.get)  
  
def select_top_loser(changes: dict[str, float]) -> str | None:  
    if not changes:  
        return None  
    return min(changes, key=changes.get)
```

Candle Builder

```
from datetime import datetime  
  
def minute_key(ts: datetime) -> datetime:  
    return ts.replace(second=0, microsecond=0)  
  
def update_candle(candle: dict | None, ltp: float, volume: int, ts: datetime) -> dict:  
    key = minute_key(ts)  
    if candle is None or candle["timestamp"] != key:  
        return {  
            "timestamp": key,
```

```

        "open": ltp,
        "high": ltp,
        "low": ltp,
        "close": ltp,
        "volume": volume,
    }
    candle["high"] = max(candle["high"], ltp)
    candle["low"] = min(candle["low"], ltp)
    candle["close"] = ltp
    candle["volume"] += volume
    return candle

```

Interview question: Why not store all ticks forever in memory?

Answer: It increases memory and GC pressure. Store latest tick, candles, and bounded recent buffers. Persist historical data separately.

4. FastAPI Basics To Advanced

FastAPI is used because it is fast, async-friendly, typed, and integrates with Pydantic.

Minimal API

```

from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class OrderRequest(BaseModel):
    symbol: str
    side: str
    quantity: int

@app.post("/orders")
def place_order(request: OrderRequest):
    return {"status": "accepted", "symbol": request.symbol}

```

Async Endpoint

```

import asyncio
from fastapi import FastAPI

```

```

app = FastAPI()

@app.get("/health")
async def health():
    await asyncio.sleep(0)
    return {"ok": True}

```

Use 'async def' for non-blocking I/O. Use thread pools for blocking SDK calls.

Background Jobs

```

from fastapi import BackgroundTasks

def warm_history(symbols: list[str]):
    for symbol in symbols:
        print("warming", symbol)

@app.post("/warmup")
def start_warmup(background_tasks: BackgroundTasks):
    background_tasks.add_task(warm_history, ["TCS", "INFY"])
    return {"status": "started"}

```

For serious trading systems, a dedicated worker queue is better than simple BackgroundTasks.

Server Sent Events For Dashboard Updates

```

from fastapi.responses import StreamingResponse
import asyncio
import json

async def event_stream():
    while True:
        yield f"data: {json.dumps({'status': 'ok'})}\n\n"
        await asyncio.sleep(1)

@app.get("/state/stream")
async def state_stream():
    return StreamingResponse(event_stream(), media_type="text/event-stream")

```

FastAPI Interview Questions

Question: What is ASGI?

Answer: ASGI is the async server gateway interface used by FastAPI through servers like

Uvicorn. It supports async HTTP and websockets.

Question: When should you not use async?

Answer: If the library is blocking, async does not help unless the blocking call is moved to a thread pool.

Question: How do you prevent heavy '/api/state' calls from slowing the system?

Answer: Cache dashboard state and update it from events. API should read cached snapshots, not recompute strategy on every poll.

5. Databases And Persistence

For a production multi-client version, use:

- * PostgreSQL for users, brokers, strategies, settings, audit logs.
- * Redis for hot state, locks, queues, and rate-limit counters.
- * Object storage for large historical candle files.

Example Tables

```
CREATE TABLE users (  
  id UUID PRIMARY KEY,  
  email TEXT UNIQUE NOT NULL,  
  password_hash TEXT NOT NULL,  
  created_at TIMESTAMP NOT NULL DEFAULT now()  
);
```

```
CREATE TABLE broker_accounts (  
  id UUID PRIMARY KEY,  
  user_id UUID REFERENCES users(id),  
  broker TEXT NOT NULL,  
  client_id TEXT,  
  access_token_encrypted TEXT,  
  active BOOLEAN DEFAULT true  
);
```

```
CREATE TABLE trades (  
  id UUID PRIMARY KEY,  
  user_id UUID REFERENCES users(id),  
  symbol TEXT NOT NULL,  
  side TEXT NOT NULL,  
  quantity INT NOT NULL,  
  entry_price NUMERIC,  
  exit_price NUMERIC,
```

```
status TEXT NOT NULL,  
created_at TIMESTAMP NOT NULL DEFAULT now()  
);
```

SQLAlchemy Model Example

```
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column  
from sqlalchemy import String, Integer, Numeric  
  
class Base(DeclarativeBase):  
    pass  
  
class Trade(Base):  
    __tablename__ = "trades"  
  
    id: Mapped[int] = mapped_column(primary_key=True)  
    symbol: Mapped[str] = mapped_column(String(30), index=True)  
    side: Mapped[str] = mapped_column(String(10))  
    quantity: Mapped[int] = mapped_column(Integer)  
    entry_price: Mapped[float] = mapped_column(Numeric)
```

Database Interview Questions

Question: What is an index?

Answer: A data structure that speeds up queries at the cost of storage and slower writes.

Question: What is a transaction?

Answer: A group of operations that commit or rollback together, preserving consistency.

Question: How would you store broker access tokens?

Answer: Encrypt at rest, restrict access, rotate regularly, never log tokens.

6. Authentication And JWT

JWT is a signed token that carries claims.

Simple JWT Flow

1. User logs in with email and password.
2. Server verifies password.
3. Server creates JWT with user ID and expiry.
4. Client sends JWT in 'Authorization: Bearer <token>'.

5. Server validates token on each request.

Example

```
from datetime import datetime, timedelta, timezone
import jwt

SECRET = "change-me"

def create_access_token(user_id: str) -> str:
    payload = {
        "sub": user_id,
        "exp": datetime.now(timezone.utc) + timedelta(hours=1),
        "iat": datetime.now(timezone.utc),
    }
    return jwt.encode(payload, SECRET, algorithm="HS256")

def verify_token(token: str) -> dict:
    return jwt.decode(token, SECRET, algorithms=["HS256"])
```

Password Hashing

Never store plain passwords.

```
from passlib.context import CryptContext

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def hash_password(password: str) -> str:
    return pwd_context.hash(password)

def verify_password(password: str, hashed: str) -> bool:
    return pwd_context.verify(password, hashed)
```

Interview answer:

Quote: JWT is good for stateless authentication, but for broker tokens I would store encrypted tokens server-side and not put broker secrets inside JWT.

7. WebSockets And Market Data

Market feed architecture:

Broker WebSocket

- > Tick normalizer
- > Candle builder
- > Strategy engine
- > Signal queue
- > Order manager
- > Dashboard cache

Normalized Tick Model

```
from dataclasses import dataclass
from datetime import datetime
```

```
@dataclass
class MarketTick:
    broker: str
    security_id: str
    symbol: str
    ltp: float
    volume: int
    timestamp: datetime
```

Subscription Manager

```
class SubscriptionManager:
    def __init__(self):
        self.current: set[str] = set()

    def diff(self, desired: set[str]) -> tuple[set[str], set[str]]:
        subscribe = desired - self.current
        unsubscribe = self.current - desired
        return subscribe, unsubscribe

    def apply(self, desired: set[str]):
        subscribe, unsubscribe = self.diff(desired)
        self.current = set(desired)
        return subscribe, unsubscribe
```

Reconnect Strategy

Use:

- * One websocket per broker account.
- * Debounced subscription changes.
- * Exponential backoff.
- * Heartbeat watchdog.
- * Do not reconnect when only watchlist changes.

Bad design:

Every stock add -> close websocket -> open websocket -> subscribe

Good design:

One connection -> batch subscribe new symbols -> keep socket alive

Interview question: Why can websocket reconnect loops happen?

Answer: Multiple duplicate connections, aggressive reconnects, blocking tick callbacks, rate limits, or not following broker subscription batch limits.

8. Algo Trading Latency

Latency matters because price can move before order placement.

Sources Of Latency

- * Broker tick delay.
- * Network delay.
- * Python processing delay.
- * Strategy calculation.
- * Order API latency.
- * Broker order queue.
- * Exchange execution.

How To Reduce Latency

- * Prewarm historical candles before market.
- * Cache instrument master and security IDs.
- * Keep websocket connected before market open.
- * Avoid REST calls during entry decision.
- * Use event queues.

- * Keep '/api/state' lightweight.
- * Use latest tick per symbol instead of processing stale tick backlog.
- * Avoid heavy logging inside tick callback.

Bounded Queue Example

```
import queue

tick_queue = queue.Queue(maxsize=10000)

def push_tick(tick):
    try:
        tick_queue.put_nowait(tick)
    except queue.Full:
        # Drop stale data rather than blocking websocket thread.
        pass
```

Latest Tick Store

```
latest_ticks: dict[str, MarketTick] = {}

def on_tick(tick: MarketTick):
    latest_ticks[tick.symbol] = tick
```

Interview answer:

Quote: In algo trading, correctness comes first, then latency. A fast system that places duplicate or wrong orders is dangerous. I separate strategy from order manager and reconcile fills before updating position quantity.

9. Trading System Architecture

Recommended architecture:

```
FastAPI App
market_data/
  dhan_feed.py
  zerodha_feed.py
  subscription_manager.py
```

```
candles/  
  candle_builder.py  
  candle_store.py  
  
strategies/  
  heuristic_stock.py  
  heuristic_nifty.py  
  heuristic_advance.py  
  hybrid_driver.py  
  
orders/  
  order_manager.py  
  dhan_orders.py  
  zerodha_orders.py  
  reconciliation.py  
  
replay/  
  replay_engine.py  
  
dashboard/  
  state_cache.py  
  serializers.py
```

Event Flow

tick -> candle -> signal -> order -> fill -> position -> dashboard

Why Separate Services?

- * Market data should not place orders.
- * Strategy should not know broker-specific payloads.
- * Order manager should not calculate indicators.
- * Replay should not touch live state.
- * Dashboard should read cached state only.

Strategy Interface

```
from dataclasses import dataclass
```

```
@dataclass  
class Signal:  
    action: str  
    symbol: str  
    reason: str
```

```
stop: float | None = None
target: float | None = None
```

```
class Strategy:
    def evaluate(self, candles) -> Signal:
        raise NotImplementedError
```

10. Order Manager And Reconciliation

Order manager is critical. It prevents duplicate orders and wrong quantity exits.

Order State Machine

```
CREATED
-> SENT
-> OPEN/PENDING
-> PARTIAL_FILL
-> FILLED
-> REJECTED
-> CANCELLED
```

Why Reconciliation Is Needed

Example problem:

- * Initial buy 10 shares filled.
- * Pyramid buy 10 shares sent but pending.
- * Strategy thinks quantity is 20.
- * Exit sends sell 20.
- * Actual demat position is only 10.

This causes over-sell or rejection.

Correct approach:

- * Position quantity should update only from broker fill updates.
- * Pending order quantity must be tracked separately.
- * Exit quantity should be actual filled open quantity.

Order Manager Example

```

class OrderManager:
    def __init__(self, broker):
        self.broker = broker
        self.pending_orders = {}
        self.positions = {}

    def enter(self, symbol: str, side: str, quantity: int):
        if self.has_pending_entry(symbol):
            return None
        order_id = self.broker.place_order(symbol, side, quantity)
        self.pending_orders[order_id] = {
            "symbol": symbol,
            "side": side,
            "quantity": quantity,
            "status": "SENT",
        }
        return order_id

    def on_order_update(self, update: dict):
        order_id = update["order_id"]
        status = update["status"]
        filled_qty = update.get("filled_quantity", 0)
        order = self.pending_orders.get(order_id)
        if not order:
            return
        order["status"] = status
        if status == "FILLED":
            self._apply_fill(order["symbol"], order["side"], filled_qty)
            self.pending_orders.pop(order_id, None)

    def has_pending_entry(self, symbol: str) -> bool:
        return any(o["symbol"] == symbol and o["status"] in {"SENT", "OPEN"} for o in
self.pending_orders.values())

    def _apply_fill(self, symbol: str, side: str, quantity: int):
        current = self.positions.get(symbol, 0)
        self.positions[symbol] = current + quantity if side == "BUY" else current -
quantity

```

Interview question: How do you avoid duplicate exit orders?

Answer: Use idempotency keys, pending-exit flags, broker order updates, and reject new exit orders while an exit is already pending.

11. Replay And Backtesting

Replay is not live trading. It must be isolated.

Correct Replay Design

Replay candles

- > replay state
- > strategy
- > simulated order manager
- > replay trade history

Live websocket should not close replay trades.

Replay Loop

```
def replay(candles, strategy):
    active_trade = None
    trades = []

    for candle in candles:
        context = {"active_trade": active_trade, "candle": candle}
        signal = strategy.evaluate(context)

        if signal.action == "BUY" and active_trade is None:
            active_trade = {"entry": candle.close, "time": candle.timestamp}

        elif signal.action == "EXIT" and active_trade is not None:
            active_trade["exit"] = candle.close
            trades.append(active_trade)
            active_trade = None

    return trades
```

Avoid Look-Ahead Bias

Bad:

```
top_gainer = full_day_close_change[symbol]
```

Good:

```
top_gainer = change_until_signal_time[symbol]
```

In hybrid replay, if NIFTY signal comes at 09:45, stock gainer/loser must be calculated using stock price available at 09:45, not day close.

12. Hybrid Mode Design

Hybrid mode:

NIFTY = driver
Stock = traded instrument

If NIFTY gives long:

- * Select top gainer from pasted stock list at that timestamp.
- * Buy that stock.

If NIFTY gives short:

- * Select top loser from pasted stock list at that timestamp.
- * Short that stock.

If NIFTY pyramids:

- * Add same quantity in stock.

If NIFTY exits:

- * Exit stock.

Hybrid Selection Example

```
def select_hybrid_stock(decision_action: str, changes: dict[str, float]) -> str:
    if decision_action == "ENTER_CALL":
        return max(changes, key=changes.get)
    if decision_action == "ENTER_PUT":
        return min(changes, key=changes.get)
    raise ValueError("Unsupported action")
```

Important Rule

Hybrid should mirror accepted NIFTY driver trades, not raw NIFTY signals.

Wrong:

Every NIFTY signal -> stock trade

Correct:

NIFTY actual trade opens -> stock trade opens
NIFTY actual add happens -> stock add happens
NIFTY actual exit happens -> stock exit happens

13. Multi-Client SaaS Design

If building this as a multi-client platform:

Requirements

- * Separate user accounts.
- * Separate broker credentials.
- * Separate websockets per broker account.
- * Strategy settings per user.
- * Audit logs.
- * Risk controls per user.
- * Token encryption.
- * Admin dashboard.

Tenant-Aware Models

Every important table must include 'user_id'.

```
CREATE TABLE strategy_settings (  
  id UUID PRIMARY KEY,  
  user_id UUID NOT NULL REFERENCES users(id),  
  mode TEXT NOT NULL,  
  settings JSONB NOT NULL,  
  updated_at TIMESTAMP DEFAULT now()  
);
```

Multi-Client Architecture

API Gateway / FastAPI

- > Auth service
- > User settings service
- > Broker connection manager
- > Per-user strategy workers
- > Order manager
- > PostgreSQL
- > Redis

Isolation

Never allow one user's:

- * broker token
- * orders
- * market feed state
- * strategy settings
- * replay result

to leak into another user's session.

Scaling

Single client:

One process can handle app + feed + strategy.

Many clients:

Separate workers by user/broker.
Use Redis streams or queues.
Use PostgreSQL for persistence.
Use Kubernetes or managed instance groups for scaling.

14. Google Cloud Deployment

GCP Services To Know

- * Compute Engine: VM hosting.
- * VPC: network.
- * Firewall rules: allow HTTP/HTTPS/SSH.
- * Static IP: fixed public IP.
- * Cloud DNS: domain mapping.
- * Cloud SQL: managed PostgreSQL.
- * Secret Manager: store secrets.
- * Cloud Logging: logs.
- * Cloud Monitoring: metrics.
- * Artifact Registry: Docker images.
- * Cloud Run: container deployment.

VM Deployment Steps

1. Create VM in 'asia-south1' Mumbai.
2. Choose Ubuntu 22.04 or 24.04.
3. Reserve static IP.
4. Attach static IP to VM.
5. Allow firewall port 80 and 443.
6. SSH into VM.
7. Clone repo.
8. Create Python venv.
9. Install requirements.
10. Create systemd service.
11. Configure Nginx reverse proxy.
12. Optional: configure SSL with Certbot if domain exists.

Commands

```
sudo -i
apt update && apt upgrade -y
apt install git python3 python3-venv python3-pip nginx -y

cd /opt
git clone https://github.com/vknowledge-123/AIsamosa.git aisamosa
cd /opt/aisamosa
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

Systemd:

[Unit]

Description=Aisamosa FastAPI app

After=network.target

[Service]

User=root

WorkingDirectory=/opt/aisamosa

ExecStart=/opt/aisamosa/.venv/bin/python -m uvicorn app.main:app --host 127.0.0.1

--port 8000

Restart=always

RestartSec=5

[Install]

WantedBy=multi-user.target

Nginx:

```
server {  
    listen 80;  
    server_name _;  
  
    location / {  
        proxy_pass http://127.0.0.1:8000;  
        proxy_http_version 1.1;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
        proxy_set_header Upgrade $http_upgrade;  
        proxy_set_header Connection "upgrade";  
    }  
}
```

Commands:

```
systemctl daemon-reload  
systemctl enable aisamosa  
systemctl start aisamosa  
nginx -t  
systemctl restart nginx
```

Logs:

```
journalctl -u aisamosa -f
journalctl -u nginx -f
systemctl status aisamosa --no-pager
systemctl status nginx --no-pager
```

Update:

```
cd /opt/aisamosa
git pull
source .venv/bin/activate
pip install -r requirements.txt
systemctl restart aisamosa
```

15. Code You Must Be Able To Write

1. Build 1-Minute Candles From Ticks

```
from datetime import datetime

class CandleBuilder:
    def __init__(self):
        self.current = {}

    def on_tick(self, symbol: str, ltp: float, volume: int, ts: datetime):
        bucket = ts.replace(second=0, microsecond=0)
        candle = self.current.get(symbol)

        if candle is None or candle["timestamp"] != bucket:
            self.current[symbol] = {
                "timestamp": bucket,
                "open": ltp,
                "high": ltp,
                "low": ltp,
                "close": ltp,
                "volume": volume,
            }
            return self.current[symbol]

        candle["high"] = max(candle["high"], ltp)
        candle["low"] = min(candle["low"], ltp)
        candle["close"] = ltp
```

```

candle["volume"] += volume
return candle

```

2. Calculate EMA

```

def ema(values: list[float], period: int) -> list[float]:
    if not values:
        return []
    alpha = 2 / (period + 1)
    result = [values[0]]
    for value in values[1:]:
        result.append((value * alpha) + (result[-1] * (1 - alpha)))
    return result

```

3. Detect Crossover

```

def crossed_above(prev_fast: float, prev_slow: float, fast: float, slow: float) -> bool:
    return prev_fast <= prev_slow and fast > slow

def crossed_below(prev_fast: float, prev_slow: float, fast: float, slow: float) -> bool:
    return prev_fast >= prev_slow and fast < slow

```

4. Top Gainer At Signal Time

```

def change_pct(latest: float, previous_close: float) -> float:
    return ((latest - previous_close) / previous_close) * 100

def select_top_gainer(prices: dict[str, float], pdcs: dict[str, float]) -> str:
    changes = {
        symbol: change_pct(price, pdcs[symbol])
        for symbol, price in prices.items()
        if symbol in pdcs and pdcs[symbol] > 0
    }
    return max(changes, key=changes.get)

```

5. Simple Stop Loss

```

def should_exit_long(current_price: float, stop_price: float) -> bool:

```

```
return current_price <= stop_price
```

```
def should_exit_short(current_price: float, stop_price: float) -> bool:  
    return current_price >= stop_price
```

6. Pyramiding

```
class Position:  
    def __init__(self, entry_price: float, quantity: int):  
        self.entry_price = entry_price  
        self.quantity = quantity  
        self.base_quantity = quantity  
        self.pyramid_count = 0  
  
    def add(self, price: float):  
        if self.pyramid_count >= 2:  
            return False  
        add_qty = self.base_quantity  
        total_cost = (self.entry_price * self.quantity) + (price * add_qty)  
        self.quantity += add_qty  
        self.entry_price = total_cost / self.quantity  
        self.pyramid_count += 1  
        return True
```

7. JWT Protected Endpoint

```
from fastapi import Depends, HTTPException  
from fastapi.security import HTTPBearer  
import jwt  
  
security = HTTPBearer()  
SECRET = "change-me"  
  
def current_user(credentials=Depends(security)):  
    try:  
        payload = jwt.decode(credentials.credentials, SECRET, algorithms=["HS256"])  
        return payload["sub"]  
    except Exception:  
        raise HTTPException(status_code=401, detail="Invalid token")
```

16. Interview Questions And Answers

Python

Question: Explain GIL.

Answer: The Global Interpreter Lock allows only one thread to execute Python bytecode at a time. For I/O-heavy apps like broker APIs, threads still help. For CPU-heavy calculations, use multiprocessing or vectorized libraries.

Question: How do you handle exceptions in trading?

Answer: Catch broker/network exceptions at adapter boundaries, record error state, retry with backoff if safe, never assume an order failed without reconciliation.

FastAPI

Question: Why FastAPI?

Answer: It is ASGI-based, supports async, Pydantic validation, automatic docs, and works well for low-latency APIs when heavy work is moved outside request handlers.

Question: How to avoid request timeout?

Answer: Return job ID immediately and run long work in background worker. Frontend polls or uses SSE for progress.

System Design

Question: Design a live trading system.

Answer:

Broker feed -> Market data service -> Candle builder -> Strategy engine
-> Order manager -> Broker order API -> Order update reconciliation
-> Position store -> Dashboard state cache

Question: Why separate order manager from strategy?

Answer: Strategy decides intent. Order manager handles execution reality: pending, partial fill, rejection, retry, quantity reconciliation.

Latency

Question: How do you reduce latency at 9:15?

Answer: Prewarm history at 9:10, cache instrument tokens, keep websocket connected, avoid REST calls at signal time, use queues, and process only latest ticks.

Question: What if websocket connects late at 9:21?

Answer: Fetch missing intraday candles from REST, merge into candle store, recalculate indicators, detect missed crossovers, then resume live ticks.

Database

Question: SQL vs NoSQL?

Answer: SQL for trades, users, settings, and audit logs because consistency matters. Redis for cache and queues.

Broker Integration

Question: How do you prevent double sell?

Answer: Track broker order status. Exit only actual filled quantity. Do not count pending pyramid as filled until order update confirms.

Leadership

Question: How would you lead a team on this project?

Answer: I would split modules: market data, strategy, order manager, dashboard, replay, DevOps. I would define interfaces, add tests for order safety, use code reviews, and prioritize risk controls before adding strategy features.

Final Interview Pitch

Quote: This project taught me end-to-end backend engineering: FastAPI APIs, broker websocket integration, historical data sync, trading strategy execution, replay simulation, paper trading, order reconciliation, dashboard state caching, and deployment on cloud servers. The biggest engineering challenge was separating live trading from replay and preventing broker execution bugs like duplicate exits or wrong quantity. If I scale it as a SaaS, I would move to PostgreSQL, Redis queues, per-client broker workers, encrypted credentials, and a proper event-driven architecture.

Practice Tasks

1. Write a candle builder from ticks.
2. Write top gainer/loser selector using previous close.
3. Write an order manager that prevents duplicate pending orders.
4. Design a FastAPI endpoint that starts replay and returns job ID.
5. Design a database schema for users, broker accounts, trades, and settings.
6. Explain how replay avoids look-ahead bias.
7. Explain how to scale from one client to 100 clients.
8. Write JWT login and protected endpoint.
9. Explain how GCP VM deployment works.
10. Explain why order reconciliation is mandatory in algo trading.

Quick Revision Checklist

- * Python OOP, dataclasses, exceptions, typing.
- * FastAPI async, BackgroundTasks, SSE, WebSocket.
- * PostgreSQL, Redis, indexes, transactions.
- * JWT, password hashing, encrypted broker tokens.
- * Broker websocket lifecycle and subscription batching.
- * Candle builder and indicator calculation.
- * Strategy vs order manager separation.
- * Replay isolation and look-ahead bias.
- * Latency reduction techniques.
- * GCP VM, static IP, Nginx, systemd, logs.